# Data Valuation

Data Valuation for Machine Learning: Experiments and Analyses [Experiment, Analysis & Benchmark]

A compact guide to the data-valuation codebase: experiment code, data processing, algorithms for measuring per-example/data-source value, and tooling for running coarse- and fine-grained evaluations.

---

## Project overview

- Purpose: research and tooling for assessing the value (importance, influence, contribution) of data at different granularities (per-example, per-source, coarse-grained groups) for ML models.
- Use cases: dataset debugging, dataset selection/prioritization, influence estimation, data summarization, experiment reproducibility and evaluation.
- What this repo contains: implementations of multiple dataval approaches, experiment harnesses, model definitions, utilities for PyTorch, and datasets and notebooks used for experiments.

Assumptions: descriptions below are inferred from file names and structure. See module docs and inline comments for implementation details.

## Quick start

1. Create a Python environment (recommended Python 3.8+).

```
python -m venv .venv
.\.venv\Scripts\activate
pip install -r requirements.txt
# Optional: editable install
pip install -e .
```

2. Run a main entry (example):

```
# Fine-grained runner (package exposes a __main__)
python -m src.fine_grained

# Run a coarse-grained script directly
python src/coarse_grained/scripts/boostrap.py

# Open notebooks for interactive exploration
jupyter lab notebooks/
```

Note: specific scripts and modules frequently accept command-line arguments; inspect the script headers or --help where implemented.

## Repository structure

Top-level layout (trimmed):

```
LICENSE
README.md
requirements.txt
setup.py
data/
experiments/
notebooks/
src/
```

- `data/` — prepared datasets used by experiments. Contains named dataset folders (e.g., `CIFAR10`, `DogFish`, `Hep`, `WebSkin_HAM`) and numpy arrays like `X_source.npy`, `y_source.npy`, `y_val.npy`.
- `experiments/` — experiment definitions and results, split into `coarse_grained` and `fine_grained` experiments, each with `evaluation/` and `tuning/` subfolders.
- `notebooks/` — interactive analysis and demos used during development and for reproducing figures or quick tests.
- `requirements.txt` and `setup.py` — dependency and packaging metadata.
- `src/` — main codebase. Primary subpackages include `coarse_grained` and `fine_grained` functionality.

Detailed `src/` layout (high level):

```
src/
  __init__.py
  coarse_grained/
    baselines/        # baseline algorithms (MMD, NTK, OT, RV, etc.)
    models/           # model definitions used in coarse experiments
    scripts/          # runnable scripts (bootstrap, robustness, etc.)
    utilities/        # plotting, utils, PyTorch helpers
  fine_grained/
    __main__.py       # fine-grained experiment entry point
    dataloader/       # dataset fetchers, noisification, registration
    dataval/          # implementations of data-valuation methods (ame, influence,
knnshap, dvrl, ...)
    experiment/       # experiment harness and utilities
    model/            # model wrappers and training code (bert, mlp,
logistic_regression, etc.)
```

## Key components and modules

- `src/fine_grained/dataval/` — implementations for many data-valuation methods. Subpackages include:
  - `ame/`, `influence/`, `knnshap/`, `margcontrib/`, `oob/`, `dvrl/`, `otg/`, `random/`, `volume/` — each appears to implement a distinct approach to estimating data importance.
- `src/fine_grained/dataloader/` — dataset fetchers and utilities. Useful for adding new datasets or applying controlled noise.

- `src/fine_grained/experiment/` — experiment orchestration: training loops, evaluation metrics, and utilities to run fine-grained comparisons.
- `src/fine_grained/model/` — model implementations/wrappers including `bert.py`, `mlp.py`, `logistic_regression.py`, `lenet.py`. These are used to train and evaluate models under different valuation methods.
- `src/coarse_grained/baselines/` — mathematical baselines and metrics for coarse-grained valuation (e.g., MMD, Sinkhorn/OT, NTK proxies).
- `src/coarse_grained/models/` — model definitions (CNN, ResNet, VGG, tabular MLP) used by coarse experiments.
- `src/coarse_grained/scripts/` — top-level runnable scripts such as `boostrap.py`, `robustness.py` and related utilities. Inspect scripts for CLI arguments and intended experiment flow.
- `src/coarse_grained/utilities/pytorch/` — a set of PyTorch-specific helpers (datasets, distance computations, networks, numerics).

Core algorithms: the codebase provides multiple strategies for scoring examples or groups (influence functions, Shapley/KNNShap approximations, model-based contributions, OT-based measures). For algorithmic details, consult the corresponding submodules in `src/fine_grained/dataval/` and `src/coarse_grained/baselines/`.

## Data

- The `data/` folder contains prepared datasets used in experiments. Notable patterns in dataset folders:
  - `X_source.npy`, `y_source.npy` — training/source data.
  - `X_val.npy`, `y_val.npy` — validation data.
  - `X_test.npy`, `y_test.npy` — test data.
- Some dataset folders include a `Readme.md` describing provenance. Always consult dataset readmes before re-running experiments.

## Running experiments and common workflows

- Inspect `experiments/*/README.md` for experiment-specific instructions and configuration (hyperparameters, dataset choices).
- Typical flow for reproducing a result:
  1. Prepare environment and install requirements.
  2. Ensure required dataset files exist in `data/` (or implement a fetcher under `src/fine_grained/dataloader/datasets`).
  3. Run a valuation method via the fine-grained runner or one of the scripts.

Example commands

```
# Run fine-grained harness (uses src/fine_grained/__main__.py)
python -m src.fine_grained --help

# Run a coarse-grained script directly
python src/coarse_grained/scripts/boostrap.py --help

# Train a model module directly (example: train an MLP if training entrypoint
```

```
exists)
python -c "from src.coarse_grained.models.mlp import train; train()"
```

Because many scripts and modules expose their own CLI, running them with `--help` is the best way to learn supported options.

## Configuration

- Global configuration and hyperparameters are mostly controlled via script arguments or within script headers. Search for `argparse`/`fire`/`click` in `src/` to find entrypoints.
- For experiments, `experiments/*/tuning/` contains parameter grids and related files.

## Notebooks

- `notebooks/` contains interactive analyses and examples. Use these to explore intermediate outputs, visualizations, and to reproduce plots from experiments.

## Tests

- There is no explicit `tests/` folder detected. Validate code by running small experiments or notebooks locally.

## Contribution

- If you want to contribute:
  - Open an issue describing the feature or bug.
  - Fork the repo and create a branch for your change.
  - Add tests or a small notebook demonstrating the change when applicable.
  - Submit a PR describing the motivation and impact.

## License & Citation

- This repository contains a `LICENSE` file at the project root. Please follow its terms when using or redistributing code.

## Notes and assumptions

- Descriptions in this README were inferred from folder names and file listings. For detailed behavior and parameter choices, consult the source files under `src/` and the per-experiment READMEs in `experiments/`.
- If you want, I can:
  - generate a short `CONTRIBUTING.md` and `CODE_OF_CONDUCT.md` template,
  - extract CLI help outputs for the main scripts and include example invocations,
  - or create a minimal runnable example (small dataset + simple valuation run).

---

If you'd like, I can refine any section (e.g., expand the API docs for `src/fine_grained/dataval/` or add step-by-step reproduction for a chosen experiment).

# Data Valuation: Fine-Grained and Coarse-Grained Evaluation

A comprehensive experimental evaluation of data valuation methods across diverse datasets, granularities, and at scale using OpenDataVal.

## Overview

This repository provides a structured framework for evaluating data valuation methods across two main granularities:

- **Fine-Grained**: Detailed, instance-level data value assessment
- **Coarse-Grained**: Aggregate, group-level data value assessment

Each granularity includes dedicated workflows for hyperparameter tuning and comprehensive evaluation.

## Repository Structure

```
data-valuation/
├── src/                              # Source code
│   ├── fine_grained/                 # Fine-grained valuation methods
│   │   ├── __init__.py
│   │   └── methods.py                # Fine-grained implementation
│   ├── coarse_grained/               # Coarse-grained valuation methods
│   │   ├── __init__.py
│   │   └── methods.py                # Coarse-grained implementation
│   └── __init__.py
│
├── experiments/                      # Experimental workflows
│   ├── fine_grained/
│   │   ├── tuning/                   # Hyperparameter tuning
│   │   ├── evaluation/               # Evaluation results
│   │   └── README.md
│   ├── coarse_grained/
│   │   ├── tuning/                   # Hyperparameter tuning
│   │   ├── evaluation/               # Evaluation results
│   │   └── README.md
│   ├── configs/                      # Experiment configurations
│   └── datasets/                     # Dataset specifications
│
├── data/                             # Data management
│   ├── raw/                          # Original datasets
│   ├── processed/                    # Processed datasets
│   └── results/                      # Experiment results
│
├── notebooks/                        # Jupyter notebooks
├── tests/                            # Unit tests
├── .gitignore                        # Python & data science config
├── requirements.txt                  # Dependencies (with opendatval)
├── setup.py                          # Package installation
```

```
├── README.md                          # Comprehensive documentation
└── LICENSE                            # MIT License
```

# Installation

## Prerequisites

- Python 3.8+
- pip

## Setup

1. Clone the repository:

```
git clone https://github.com/validal/data-valuation.git
cd data-valuation
```

2. Create a virtual environment:

```
python -m venv venv
source venv/bin/activate  # On Windows: venv\Scripts\activate
```

3. Install dependencies:

```
pip install -r requirements.txt
```

4. Install the package in development mode:

```
pip install -e .
```

# Usage

## Fine-Grained Experiments

For instance-level data valuation using OpenDataVal:

```
from src.fine_grained import methods

# Your fine-grained valuation code here
```

Experiments and results go in:

- `experiments/fine_grained/tuning/` - Tuning experiments
- `experiments/fine_grained/evaluation/` - Evaluation results

### Coarse-Grained Experiments

For group-level data valuation using OpenDataVal:

```
from src.coarse_grained import methods

# Your coarse-grained valuation code here
```

Experiments and results go in:

- `experiments/coarse_grained/tuning/` - Tuning experiments
- `experiments/coarse_grained/evaluation/` - Evaluation results

## Data Organization

- **`data/raw/`**: Store original, unmodified datasets
- **`data/processed/`**: Store cleaned and processed datasets
- **`data/results/`**: Store evaluation results and metrics

## Datasets

Dataset specifications and metadata should be placed in `experiments/datasets/`.

## Framework

This project uses OpenDataVal as the main framework for data valuation methods.

## Contributing

Contributions are welcome! Please feel free to submit pull requests or open issues for bugs and feature requests.

## License

This project is licensed under the MIT License - see the LICENSE file for details.

## Citation

If you use this repository in your research, please cite it appropriately:

```
@repository{validal2026datavaluatione,
  title={Data Valuation: Fine-Grained and Coarse-Grained Evaluation},
  author={Validal},
  year={2026},
```

```
  url={https://github.com/validal/data-valuation}
}
```

## Contact

For questions or inquiries, please open an issue on the repository.